arXiv:2611.10315v1 [cs.PF] 6 Nov 2026

REPEATING WORK CAN LOWER ENERGY PER SUCCESSFUL GOAL IN LLM AGENTS

Deepak Panigrahy [1] **Stephen Abkin** [1] **Aakash Tyagi** [1] **David Kebo Houngninou** [1] **Rabi Mahapatra** [1]

ABSTRACT

When a temporary call failure interrupts a tool-using LLM agent, the framework must choose how much completed work to repeat. Repeating less completed work intuitively saves computation, but recovery mechanisms also change subsequent execution and the chance of success. We compare recovery mechanisms by energy per successful goal (EpG), which divides the energy of all task runs (including failed runs) by the number that finish correctly. In controlled vLLM experiments on one node, we measure board energy above idle for eight recovery options on reliably solved multi-turn tasks, with selected comparisons across three model families. For the model tested across all eight options, with the prefix cache retained, starting over has 22 percent lower EpG than continuing after early failures. Under the same cache condition, continuing has 24 percent lower EpG than starting over after late failures. Among all eight recovery options, repeating only the interrupted turn has the lowest observed EpG after early and late tool failures on this model. Holding recovery fixed, retaining the prefix cache reduces prompt tokens needing fresh computation by 86 percent but reduces recovery energy by only 8 percent at contexts up to 7,333 tokens. These measurements and a condition relating recovery energy and success show when repeating work lowers EpG, helping framework developers choose how their agents recover on their intended models and tasks.

1 INTRODUCTION

Tool-using LLM agents manage files and book travel through sequences of model and tool calls, so a failed call can interrupt a task after substantial work. We study transient failures (i.e., temporary failures of a model or tool call that clear so execution can continue). The framework running the agent must then decide how much completed work to repeat. Repeating less work avoids immediate recomputation, but the recovery also changes how execution continues and whether the task finishes correctly. Which recovery uses the least energy for a correct result?

Figure 1 compares recoveries by the conversation they retain and the tool state they restore (i.e., the data maintained by the tools). Restarting begins the task again from the first user message and resets the tools to their initial state. To repeat only the interrupted turn (i.e., the calls serving one user message), rollback restores both conversation and tools to that turn's beginning. Resuming keeps the conversation and the current tool state and continues with an error notice (Wu et al., 2023; LangChain, 2024); for the clean failures, the failed call leaves the tools as they were just before it. Grounded resume adds the actual state of the tools touched by the interrupted turn to that notice.

Resuming preserves completed work, and the server can also reuse model state already computed for the beginning of a prompt. A prefix cache stores this state, so keeping the cache warm (i.e., retaining the computed state for reuse during recovery) avoids processing much of the earlier conversation again (OpenAI, 2025; Anthropic, 2025). Retaining the conversation also affects what the agent does next, and a failed step kept in that conversation can lead to repeated calls or incorrect results (Yang, 2026; Gumaan, 2026). For a developer deciding how agents should recover, preserving completed work is useful when it lowers the energy needed for correct task completion.

To compare recoveries on that basis, we include the energy spent on unsuccessful runs. A goal is one task run, including its initial execution and any recovery. Mean energy per run divides total energy by the number of runs, while energy per successful goal (EpG) divides the same total by the number that finish correctly (Panigrahy et al., 2026b). A recovery that uses less energy per run can still require more energy per successful goal if it succeeds less often. Table 1 illustrates this difference after a late tool error. Resuming uses the least energy per run, while rollback completes more runs correctly than resuming and has the lowest energy per successful goal.

[1]Texas A&M University, College Station, TX, USA. Correspondence to: Deepak Panigrahy <deepakp@tamu.edu>.

Preprint.

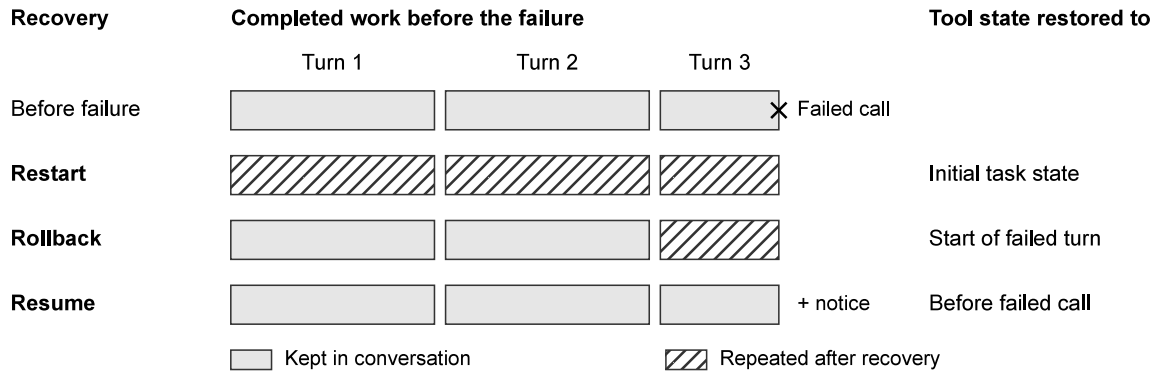

Figure 1. **What each recovery keeps and repeats.** A failure interrupts turn 3. The blocks represent work completed before the failure, grouped by user turn, with a partial block for turn 3. Gray blocks remain in the conversation, and hatched blocks show work to repeat. The right column identifies the earlier state to which the tools are restored. Resume adds an error notice. Grounded resume also includes the tools' actual state in that notice. We separately retain (warm) or clear (cold) the prefix cache for each recovery.

Table 1. **Energy per run and energy per successful goal favor different recoveries.** Qwen3 30B, 30 tasks, each interrupted by one late tool error. Each run includes its recovery. Mean run energy averages over successful and failed runs. Energy per successful goal (EpG) divides their total energy by the number that finish correctly. Grounded resume adds the tools' actual state to the error notice supplied when resuming. All four recoveries use a warm prefix cache. Bold marks the lowest observed EpG among these recoveries.

	Restart	Resume	Grounded resume	Rollback
Correct runs	28/30	24/30	26/30	27/30
Mean run energy, J	1,711	1,208	1,337	1,295
Energy per successful goal (EpG), J	1,833	1,510	1,543	**1,439**

Motivation. In production telemetry from thousands of organizations, about one in twenty model requests failed in February 2026 (Datadog, 2026). A framework's choice of recovery is reused across tasks and agents (Wu et al., 2023; LangChain, 2024), so its energy consequences accumulate over repeated failures. Developers can change how much work recovery repeats while keeping the model and hardware fixed.

Goal. We seek to determine when repeating completed work reduces the energy needed for correct task completion. Recovery research develops ways to detect failures, restore execution state, and choose a recovery point, and evaluates whether the recovered task finishes correctly (Yu et al., 2026; Zhang & Liu, 2026; Zhuang et al., 2026; Dubey, 2026). Serving research manages prefix caches to reduce latency, price, or carbon cost (Abhyankar et al., 2024; Pan et al., 2025; Khailo, 2026; Tian et al., 2026), and agent energy studies characterize where an agent's energy goes (Fortuna et al., 2026; Luo et al., 2026). The missing comparison is how these recovery choices affect measured energy per successful goal when both prefix-cache reuse and unsuccessful runs are included. To address this gap, we compare the four recovery operations with the prefix cache retained or cleared, giving eight options. We test them after matched failures (i.e., failures of the same type injected at the same planned call number). Moreover, we ask four research questions in which we answer in Section 5:

RQ1: When does restarting the task use less energy per successful goal than resuming from the failed step?

RQ2: How does repeating only the interrupted turn compare with restarting the task or resuming from the failed step?

RQ3: How much recovery energy does a warm prefix cache save when the recovery is held fixed?

RQ4: Can information available when a failure occurs guide recovery choices that reduce energy per successful goal on new tasks?

We select multi-turn tasks from the Berkeley Function Calling Leaderboard (BFCL) (Patil et al., 2025) that the model completes reliably without injected failures. Its checker determines whether a run succeeds by comparing the tools' final states after each turn with the expected states. To answer RQ1, we inject one transient failure early or late in each run and compare restarting with resuming. We add rollback to answer RQ2, and we retain or clear the prefix cache for each recovery to answer RQ3. We test all eight options on Qwen3 30B, our main model, across 30 tasks (the main study), and selected options on two other model

families. To answer RQ4, we fit decision rules on earlier experiments and execute the recoveries they choose as agents run 28 additional tasks excluded from fitting. We serve the models with vLLM (Kwon et al., 2023) and measure energy at the board of one GB10 node while running one goal at a time, using its power with the model loaded and no requests as the idle baseline.

Summary of findings. In the main study, restarting uses less energy per successful goal than resuming after early failures, while resuming uses less than restarting after late failures (§5.1). Restarting's early advantage reflects its higher success rate and the energy resuming uses above the estimated energy of unfinished work. Our cost model weighs these contributions against the energy already spent, which is larger after late failures. After early failures, resuming succeeds less often across the three tested model families, with the energy ranking also varying by model and failure type. After tool failures in the main study, repeating only the interrupted turn reduces energy per successful goal relative to resuming early and restarting late (§5.2). Across the main study's failure types and points, warm rollback has the smallest mean excess cost among the fixed recoveries tested. At prompt lengths up to 7,333 tokens, a warm prefix cache reduces input tokens requiring fresh computation by 86 percent but reduces energy used after failure by only 8 percent (§5.3). On tasks excluded from fitting, the token rule using the failure type and the number of calls already made reduces energy per successful goal relative to always resuming and always restarting (§5.4). It mostly chooses rollback with a warm prefix cache, and its estimated saving over always rolling back cold is within the variation across tasks. We propose that developers choose how their agents recover by measuring energy per successful goal for the models and tasks those agents will run.

Contributions.

- We characterize the energy per successful goal of eight recovery options and show when repeating work lowers it, using 30 multi-turn tasks with selected comparisons across three model families (§4, §5).
- We build a setup for measuring recovery energy against verified task outcomes and release the recovery decision library (Panigrahy et al., 2026a) (§4).
- We derive a condition for when restarting lowers energy per successful goal and evaluate its terms from measured executions (§3, §5.1).

2 Background and Related Work

Agent execution and call failures. A tool-using agent completes a task by asking a model what to do next, executing the requested tools, and adding their results to the conversation (Yao et al., 2023; Schick et al., 2023; Patil et al., 2023). Each model call receives the conversation so far as its input prompt, which grows as replies and tool results are added. In the road-trip task of our study, six user requests take the agent from looking up zip codes to starting the engine. Completing those requests requires 18 model calls and 13 tool calls.

Because execution depends on both model responses and tool results, a failure on either side interrupts the agent's progress. We study four call failures (Section 4). On the model side, a request is rejected by a rate limit or the reply is malformed so the framework cannot parse its tool call. On the tool side, a call returns an error or reaches its timeout without a response. Automatic retries by a model client or tool library may resolve a failure before the agent needs to recover. Our comparison concerns failures passed to the agent framework, where it must choose between continuing from retained progress and repeating earlier work.

Restoring execution after a failure. Checkpointing saves a program's execution state so it can return to that state after a failure. Classical models choose how often to save state by balancing the cost of saving it against the work lost when failures occur (Young, 1974; Daly, 2006). For an agent, recovery also determines which conversation messages and tool data will guide its subsequent actions. Rollback-Induced Reflection restores an earlier state while retaining lessons from the failed execution (Yu et al., 2026). Zhang & Liu (2026) check the conditions for reusing saved progress before execution continues. AgentRewind restores saved versions of both the agent's conversation and its workspace, and tests their roles by disabling each restoration separately (Zhuang et al., 2026). Our rollback also restores conversation and tool data together, so its measured energy and success describe the combined recovery operation.

After execution is restored, the model's next decision still depends on the messages remaining in the conversation. Yang (2026) analyzes retries under the assumption that a failed attempt left in the conversation increases the chance of further errors. In experiments with small models, Gumaan (2026) finds that showing a failed call and its error message increases the likelihood of repeating that call. For agents with persistent memory, Wang et al. (2026b) trace how faulty memories affect later decisions and selectively repeat affected work while preserving information supported by independent evidence. We compare recovery choices by measured board energy per successful goal, varying cache retention to account for the prompt computation the server reuses.

Reusing prompt computation during recovery. A model call processes its input prompt during prefill and

then generates the reply one token at a time during decoding (Zhong et al., 2024; Patel et al., 2024). With prefix caching, an inference engine such as vLLM reuses computed state for the beginning of a matching prompt, reducing the work needed for prefill (Kwon et al., 2023; Zheng et al., 2024; vLLM Project, 2025). Cached input tokens reuse this state, while uncached prompt tokens require fresh prefill. Output tokens are generated during decoding, and context length is the number of input tokens supplied to a model call. A warm recovery retains the prefix cache, while a cold recovery starts after it has been cleared. Both supply the conversation specified by the recovery operation, so retaining conversation history and reusing its computed state are separate choices.

Serving systems manage prefix caches according to their expected reuse to reduce latency (Abhyankar et al., 2024; Luo et al., 2025; Pan et al., 2025; Li et al., 2026a; Kang et al., 2026). Other studies evaluate the provider charges for keeping a prompt cached through tool pauses (Khailo, 2026), or choose what to cache according to carbon cost, as in GreenCache (Tian et al., 2026). The energy of a model call also depends on model size, output length, batch size, context length, and inference phase (Samsi et al., 2023; Luccioni et al., 2024; Chung et al., 2025). Wu et al. (2026b) analyze how prompt processing, reply generation, context length, and batching affect energy. Measured recovery energy covers input processing, output generation, tool execution, and waits, so reusing prompt computation saves only part of that energy. We measure that contribution by keeping the recovery operation fixed and comparing its retained-cache and cleared-cache versions.

Energy and verified task completion. Evaluating accuracy and cost together shows what an agent spends to produce useful results (Kapoor et al., 2024). Energy per successful goal includes the energy of failed runs in the total charged to correct completions (Panigrahy et al., 2026b). OSWorld-Human follows a related principle for computer-use agents, accounting for unnecessary steps and work spent on failed tasks (Abhyankar et al., 2026). It identifies rollback and prefix caching as promising ways to improve efficiency. We measure how these choices affect energy per successful goal after controlled call failures.

Other approaches stop agents likely to fail (Pham et al., 2026; Ruan et al., 2026; Wang et al., 2026a), detect failures from measurements collected during execution (Dubey, 2026), or measure tokens spent after an early warning of failure (Li et al., 2026b). At the infrastructure level, systems control GPU power and schedule model execution (You et al., 2022; Patel et al., 2023; Chung et al., 2024), while fault-tolerant serving systems restore interrupted computation (Strati et al., 2024; Cao et al., 2026). Comparing recoveries requires accounting for the energy already spent before failure and the energy of runs that finish incorrectly. We next use this accounting to derive when repeating completed work reduces the energy needed for a correct result.

3 Energy per Successful Goal

For our BFCL tasks, the benchmark's automated checker determines success by comparing the tools' states after each user turn with the expected states. An attempt is one execution segment, ending at the injected failure or when execution finishes. A goal is one task run, including the initial attempt and any recovery attempts. Goal energy sums the measured energy of those attempts, while recovery energy counts only the attempts after the initial failure. We specify what is metered in Section 4.

Mean energy per run divides total goal energy by the number of runs, including successful and failed runs. For a group with at least one correct goal, energy per successful goal (EpG) divides that same total by the number of correct goals, giving joules per correct completion (Panigrahy et al., 2026b). In Table 1, warm resume averages 1,208 J per run and completes 24 of 30 goals correctly, giving approximately $1{,}208/(24/30) = 1{,}510$ J per successful goal. Warm rollback averages 1,295 J per run and completes 27 of 30 goals correctly, giving $1{,}295/(27/30) \approx 1{,}439$ J per successful goal. Rollback's higher success makes it cheaper per correct completion despite its higher energy per run.

When repeating work reduces energy. We now compare restarting with resuming from the same state at the failure. Let S be the energy already spent. The experiments approximate this comparison with separate executions, checking agreement of their calls and replies before the failure (Section 4). For a recovery o, let R_o be its expected recovery energy, and let $p_o > 0$ be its probability of completing the goal correctly. This expectation averages the energy of both successful and failed recoveries. The expected goal energy is $S + R_o$, so the energy per successful goal is

$$J_o = \frac{S + R_o}{p_o}. \tag{1}$$

Dividing by p_o charges more of the pre-failure energy to each correct completion when recovery succeeds less often.

Let E be the expected energy of a restart's recovery attempt, and let $p_R > 0$ be its probability of completing the goal correctly. Because restarting repeats the task, our simplified cost model uses $E - S$ to estimate the energy of unfinished work. Resuming has expected recovery energy $E - S + \Delta$ and succeeds with probability $p_K > 0$. The residual Δ, which we call additional recovery energy, is positive when resuming uses more energy than the estimate $E - S$. Restarting's goal energy is $S + E$, while resuming's is $S + (E - S) + \Delta = E + \Delta$. Substituting these totals into

Equation 1, we find that restarting is cheaper exactly when $(S + E)/p_R < (E + \Delta)/p_K$. Multiplying by the positive success probabilities and rearranging, we obtain

$$S < \left(\frac{p_R}{p_K} - 1\right)E + \frac{p_R}{p_K}\Delta. \quad (2)$$

On the right, the success-rate term $(p_R/p_K - 1)E$ accounts for the difference in success rates, while the additional-energy term $(p_R/p_K)\Delta$ weights the residual by p_R/p_K. Restarting is cheaper when their sum exceeds the energy already spent, S.

We estimate S by averaging the mean energies before failure for restart and resume, and estimate E from mean recovery energy when restarting. We compute Δ as the mean recovery energy when resuming minus $E - S$ (Appendix C). We use the same estimate of energy spent before failure for both recoveries. Attributing Δ to particular actions requires a separate comparison.

With equal success rates, Equation 2 reduces to $S < \Delta$. Restarting then saves energy only when resuming uses more energy above the estimate than has already been spent. With $\Delta = 0$, a higher restart success rate can still make restarting cheaper when $S < (p_R/p_K - 1)E$. Success rates enter alongside the repeated computation studied in classical checkpoint models (Young, 1974; Daly, 2006). Holding the other terms fixed, a smaller S favors restarting, which motivates comparing failures early and late in a task.

For rollback, let T be the energy of the completed part of the interrupted turn that must be repeated, and let $p_B > 0$ be rollback's success probability. Assuming the remaining work has expected cost $E - S$, goal energy is $E + T$ and $J_B = (E + T)/p_B$ (Appendix C). We compare restarting with resuming after early and late failures (Section 5.1), then evaluate rollback's energy and correctness (Section 5.2).

4 Experimental Design

We compare how much energy each recovery uses to complete a task correctly after the same planned failure. Changing the recovery varies how much work the agent repeats, while retaining or clearing the prefix cache varies how much prompt computation the server reuses. Early and late failures change how much work is already complete. The benchmark checks correctness independently of the agent's report. We measure the same parts of execution for each recovery and include energy from failed goals, supplying the terms in Equation 2 (Figure 2).

Models and tasks. The main study uses Qwen3 30B A3B Instruct 2507 in FP8 (Yang et al., 2025), served with vLLM (Kwon et al., 2023) and prefix caching enabled. We check whether the recovery ordering recurs in other model families using Mistral Small 3.2 24B in 8 bit (Mistral AI, 2025) and gpt-oss 20B in MXFP4 (OpenAI et al., 2025). Each model keeps its representation fixed across recovery comparisons and runs on tasks it completes reliably. The additional families are tested with warm restart, warm resume, and cold rollback. A Qwen3 4B pilot compares restart and resume on 10 tasks.

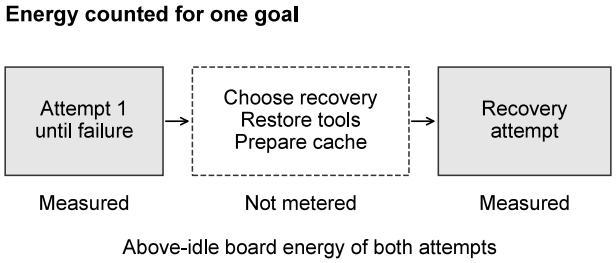

Figure 2. **One goal includes the initial attempt and recovery.** One planned call failure interrupts the first attempt. The gray boxes show the measured attempts, whose energies are summed for the goal. Choosing the recovery, restoring tools, and preparing the cache occur in the dashed box between attempts and are unmetered. Boxes represent execution stages, and arrows give their order.

To study failures that disrupt otherwise solvable tasks, we select from the BFCL multi-turn base set (Patil et al., 2025; Yan, 2025), whose expected tool states are known. Screening requires three successful runs without injected failures and the same numbers of model calls and tool calls in those runs. We also require at least 8 model calls and 5 tool calls, leaving room to inject failures at an early and a late point. Of 198 base tasks, 58 meet these conditions for the main model. We use 30 for the main comparison and reserve the other 28 for testing decision rules on additional tasks. The selected tasks involve vehicle controls, files, trading, travel bookings, messaging, and ticket management, with 14 to 31 model and tool calls combined in a run without injected failures. Separate reference runs without injected failures succeed in 84 of 90 goals. These are distinct from the screening runs. Study counts for each experiment are given in Appendix A, Table 3. Runs in the main study keep decoding settings fixed at temperature 0 and model seed 0. We also sample responses at temperature 0.7 to check whether the recovery ordering persists when token selection varies.

Failures and recoveries. We distinguish model-side failures (a malformed reply or rate-limit rejection) from tool-side failures (a tool error or timeout). An early failure occurs at the second model call or second tool call, according to the side affected. For a late failure, we set the call number near 70 percent of the corresponding model-call or tool-call count in the task's first failure-free screening run, before running the recovery comparisons. The failure occurs once, during the initial attempt. A goal enters a comparison when its log confirms that the planned failure occurred once at its

target (Appendix A). Each goal allows one recovery attempt, and each attempt has a limit of 40 model calls.

The main study compares restart, resume, rollback, and grounded resume, each with the prefix cache retained or cleared. We randomize the order within two measurement batches (Mytkowicz et al., 2009), first comparing restart with resume, then rollback with grounded resume on the same tasks and failure plan. The four failure types at two positions give eight failure conditions per task. With four operations and two cache states, this gives 1,920 goals with injected failures across the 30 main-study tasks, plus 90 reference goals without injected failures. Each recovery has a separate initial execution, so we check how closely those executions agree before failure. We replace generated call and request identifiers with common labels before comparing the calls and replies. Within the restart/resume batch, these histories agree in 238 of 240 task/failure/position combinations. Within the rollback/grounded-resume batch, they agree in 240 of 240 combinations (Appendix A).

The two model-side failure types give the same correctness outcome in 98.5 percent of paired goals, and the two tool-side types agree in all paired goals. We report grouped results by model side or tool side, pooling goals across the two types where stated. A separate study examines wrong actions that a tool accepts, extending the comparison to misleading steps that leave no error notice.

Energy measurement. A-LEMS (Panigrahy et al., 2026b) measures power at the node's DC input, covering memory and power conversion as well as the GPU (Luo et al., 2026). We sample board and chip-package power at 10 Hz and GPU power at 1 Hz. The GB10 node pairs a Grace CPU and a Blackwell GPU in 128 GB of shared memory (NVIDIA, 2025). We serve one goal at a time, so each attempt window holds only that goal's work, including a rate limit's wait. We subtract the power drawn with the model loaded and no requests (47.8 W for the first batch, remeasured for the second) to measure energy above the idle baseline. We measure each initial and recovery attempt, including waits within an attempt. Choosing a recovery, restoring tools, and preparing the cache occur between these windows and are unmetered (Figure 2). To quantify the excluded intervals, the main study records median gaps of 0.15 s with a warm cache and 0.17 s with a cold one, about 1 percent of a goal's attempt time and nearly equal across recoveries. If these intervals drew the same mean above-idle power as the attempts, they would add about 1 percent to measured goal energy at the median and less than 2 percent for 95 percent of main-study goals.

Pairing and uncertainty. We calculate each percentage saving from the two recoveries' pooled EpGs on the tasks included in both. We obtain 95% confidence intervals (CIs) from 2,000 resamples of whole tasks with replacement, using the same task draw for both recoveries and all included failure conditions (Efron & Tibshirani, 1994). This retains the dependence among outcomes from the same task. Tables report EpG for their stated populations. Held-out savings use 27 tasks with all four failure conditions (108 goals per policy), while Appendix D, Table 6 reports all 111 goals per policy. For the cache comparison, we instead calculate the reduction in pooled recovery energy or uncached prompt tokens and resample tasks in the same way. An observed minimum identifies the cheapest recovery in that sample. We describe a saving as beyond the variation across tasks when its paired interval excludes zero.

Decision rules. The main comparisons identify low-energy recoveries after observing their outcomes. We next test whether those measurements support choosing a recovery when the failure occurs, using rules fitted before evaluation on the 28 held-out tasks. The threshold rule uses the failure type and total number of model and tool calls already made, restarting below a fitted cutoff and resuming above it. The token rule uses the same information to predict each option's uncached prompt tokens after failure and selects the smallest prediction. Each option has its own fit, reflecting the input needed across the calls generated during recovery. The learning controller updates distributions of success probabilities from observed outcomes. Using Thompson sampling (Thompson, 1933), it samples success rates and chooses the smallest predicted recovery energy divided by the sampled rate. This score uses recovery energy, while Equation 1 also includes energy spent before failure. Below its fitted cutoff, its available actions were limited to restart (Appendix D).

We compare the three rules with applying warm restart, warm resume, or cold rollback at every failure, and use recovery policy for either a rule or one of these fixed choices. For the main model, each policy executes the recovery it selects after early and late malformed replies and tool errors on the held-out tasks, giving 111 decisions per policy. The threshold rule and learning controller use an earlier calibration detailed in Appendix D. One task's late tool-error case is excluded because its injection missed the planned call.

5 Results

We compare how much work recovery should repeat, how much energy cache retention saves, and whether information available at failure guides a choice on new tasks. Results are for the main study unless stated, and a positive saving means lower energy per successful goal for the first recovery named. The recoveries reported in Appendix B, Table 4 cost 1.2 to 2.0 kJ per successful goal.

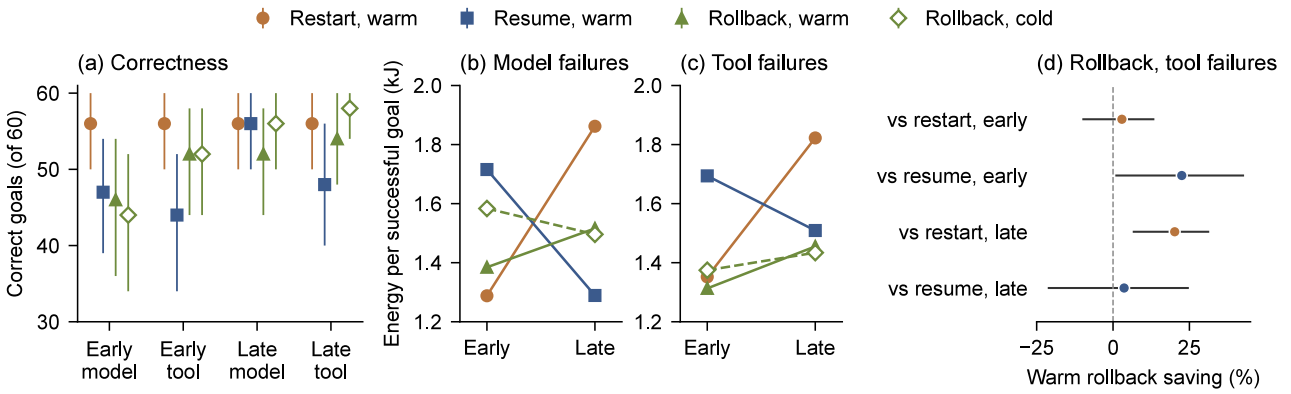

Figure 3. **Restarting is cheaper early, resuming is cheaper late, and rollback has the lowest observed EpG after tool failures.** Main study, Qwen3 30B. (a) Correct goals out of 60 for each failure side and position, with 95% bootstrap intervals over the 30 tasks. Each task contributes two failure types. (b,c) Arithmetic means of the two failure-type EpG values within each side, in kJ measured at the board. Lines connect the two measured positions. (d) Paired saving in EpG of warm rollback over warm restart and warm resume after tool failures, with 95% bootstrap intervals over tasks. Positive values favor rollback.

5.1 RQ1: When Repeating Work Lowers Energy per Successful Goal

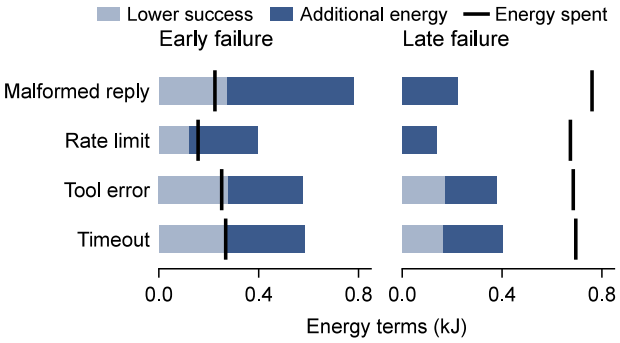

Figure 4. **Lower success and additional recovery energy account for the early resume's cost.** Main study, warm restart against warm resume, for each failure type early (left) and late (right). Bars stack the two terms on the right of Equation 2, the cost of lower success (light) and additional recovery energy (dark), measured relative to the estimated unfinished work and weighted by p_R/p_K. The black line marks the energy already spent at the failure. Restarting lowers EpG where the bar exceeds the line.

Restarting lowers energy per successful goal after early failures and resuming lowers it after late ones, both beyond the variation across tasks (Figure 3). After an early failure, restarting repeats little completed work, making it easier for a higher success rate to offset that repetition. With the cache warm, restarting completes 112 of 120 goals correctly and resuming completes 91 of 120 goals correctly after early failures (Figure 3a). Restarting matches the failure-free reference success rate. Restarting uses 22 percent less energy per successful goal than resuming (95% CI 9 to 35 percent). Equation 2 accounts for this difference through the success rates and the energy used after failure (Figure 4, Appendix C). Across failure types, its success-rate term is 120 to 280 J and its additional-energy term is 280 to 510 J. Their sum exceeds the 160 to 270 J already spent, satisfying the condition for restarting to use less energy per successful goal.

After late failures, the energy already spent is 670 to 760 J, exceeding the 140 to 400 J sum of the two terms (Figure 4). A late restart repeats most of the task, while after model-side failures both resume and restart complete 56 of 60 goals correctly. Resuming uses 24 percent less energy per successful goal than restarting after late failures (95% CI 10 to 36 percent). After late tool failures, restarting still succeeds more often, but the success advantage is too small to offset repeating the work. Time per successful goal follows the same early–late ordering (Appendix B).

Correctness across models and sampled decoding. Restarting's early correctness advantage also appears under sampled decoding and on the additional models. At temperature 0.7, resume completes 47 of 60 goals correctly and restart completes 58 of 60 goals correctly. For Mistral, resume completes 26 of 32 goals correctly and restart completes 31 of 32 goals correctly. For gpt-oss, resume completes 29 of 48 goals correctly and restart completes 43 of 48 goals correctly. For Qwen3 4B, resume completes 24 of 40 goals correctly and restart completes 40 of 40 goals correctly. For Mistral after early tool failures, resuming fails one of eight goals for each failure type, and its energy per successful goal is lower than restarting's (Appendix B, Table 5). For these tasks and failures, restarting's higher success rate is too small to offset its higher measured goal energy. In a separate study of wrong steps that tools ac-

cept without an error notice, excluding self-corrected cases leaves a different population for each recovery. Warm resume completes 3 of 13 early goals correctly and warm restart completes 12 of 13 early goals correctly in their respective retained populations (Appendix A).

Finding 1. On the main model, restarting uses 22 percent less energy per successful goal than resuming after early failures. The resume's lower observed success rate and additional recovery energy account for this gap. After late failures, the energy already spent exceeds those two terms together, and resuming uses 24 percent less energy per successful goal than restarting (Figures 3 and 4).

5.2 RQ2: How Rollback Compares with Restart and Resume

After tool failures on the main model, repeating only the interrupted turn lowers EpG relative to resuming early and restarting late. The estimated savings over restarting early and resuming late are within the variation across tasks (Figure 3d). Rollback repeats only the interrupted turn, allowing us to test whether restoring more state than resume while repeating less work than restart lowers energy per successful goal. On the main model, rollback had the lowest observed energy per successful goal of the eight recovery options after tool failures, both early and late (Appendix B, Table 4). For each type of late tool failure, warm rollback completes 27 of 30 goals correctly and cold rollback completes 29 of 30 goals correctly. Warm grounded resume completes 26 of 30 goals correctly and warm resume completes 24 of 30 goals correctly (Table 1 and Appendix B, Table 4). After early tool failures, warm rollback uses 2.9 percent less energy per successful goal than warm restart (95% CI −9.7 to 13.2 percent) and 22.5 percent less energy per successful goal than warm resume (1.1 to 42.6 percent). After late tool failures, warm rollback uses 20.2 percent less energy per successful goal than warm restart (6.8 to 31.2 percent) and 3.6 percent less energy per successful goal than warm resume (−21.0 to 24.5 percent). Rollback ran in the second batch, where paired initial attempts used about 5 percent more energy. Dividing second-batch energy by this ratio, assuming it also applies to recovery, preserves the observed minima and the direction of the resolved differences in this section (Appendix A).

Rollback restores the conversation and tools to the start of the interrupted turn. Resume keeps the error notice and restores tools only to just before the failed call. The measured energy difference describes that combined operation. After late tool failures, resuming completes 14 of 16 goals correctly for Mistral and 23 of 24 goals correctly for gpt-oss, and has lower energy per successful goal than the tested alternatives (Appendix B, Table 5).

Choosing one fixed recovery. A framework using the same recovery for every failure needs to know the extra cost of that choice. On the main model, warm rollback uses on average 6.5 percent more energy per successful goal than the cheapest recovery for each failure type and point (at most 18 percent, after late rate limits). The mean excess cost is 10.1 percent for cold rollback, 15.2 percent for warm resume, and 18.7 percent for warm restart. Warm rollback has the smallest mean excess cost among the fixed recoveries tested and repeats at most one turn of work (Appendix D, Table 7).

Finding 2. On the main model, repeating only the interrupted turn gives the lowest observed EpG after tool failures, both early and late. Warm rollback uses 22.5 percent less energy per successful goal than warm resume early and 20.2 percent less energy per successful goal than warm restart late, beyond the variation across tasks. The estimated savings over warm restart early and warm resume late are within the variation across tasks. Across all eight failure conditions, warm rollback has the smallest mean excess cost among the fixed recoveries tested (Figure 3d, Appendix D, Table 7).

5.3 RQ3: How Much Recovery Energy Cache Retention Saves

At the task context lengths, cache retention removes most uncached prompt tokens but changes recovery energy much less (Figure 5). To measure the effect of a warm prefix cache, we compare warm and cold versions of the same recovery operation, allowing their subsequent calls and outcomes to differ (Section 2). At task contexts up to 7,333 tokens, a warm cache removes 86 percent of a recovery's uncached prompt tokens (95% CI 85 to 87 percent) but only 8 percent of its energy (6 to 10 percent, Figure 5a).

Uncached prompt tokens measure input that requires fresh prefill. Recovery energy also covers output generation, tool execution, and waits, so reusing prompt computation saves only part of that energy. Cache retention nevertheless helps in most measured conditions. The cheapest recovery runs warm in six of the eight pairs of failure type and point, with the two cold winners within 2 percent of their warm versions.

The relative energy saving from caching grows with context in the isolated-call sweep (Figure 5b). On the same node, without tool time or waits, a cached call costs 0.54 of the energy of an uncached call at 4k input tokens. The ratio falls to 0.10 near 32k input tokens. Over the reported fits, uncached-call energy grows faster than linearly while cached-call energy grows more slowly (Appendix A).

Tokens per successful goal divide the input and output tokens of all goals by the number that finish correctly, with cached input counted once. Among warm restart, resume,

Table 2. Tokens and energy per successful goal for warm recoveries in the main study. Token counts are in thousands and include failed goals. Total counts input plus output, with cached input counted once as part of input. Unc. counts input tokens requiring fresh computation. EpG gives board energy per successful goal in kJ. Side identifies model-call or tool-call failures. Bold marks the lowest EpG in a row.

		Restart			Resume			Rollback		
Side	Point	Total	Unc.	EpG	Total	Unc.	EpG	Total	Unc.	EpG
Model	early	56.0	4.4	**1.29**	69.9	5.3	1.70	66.7	5.2	1.38
Tool	early	59.3	4.4	1.35	72.0	5.6	1.69	59.6	4.7	**1.31**
Model	late	82.1	4.7	1.86	56.4	4.4	**1.29**	63.4	4.8	1.52
Tool	late	80.7	4.6	1.82	65.1	5.2	1.51	61.3	4.6	**1.45**

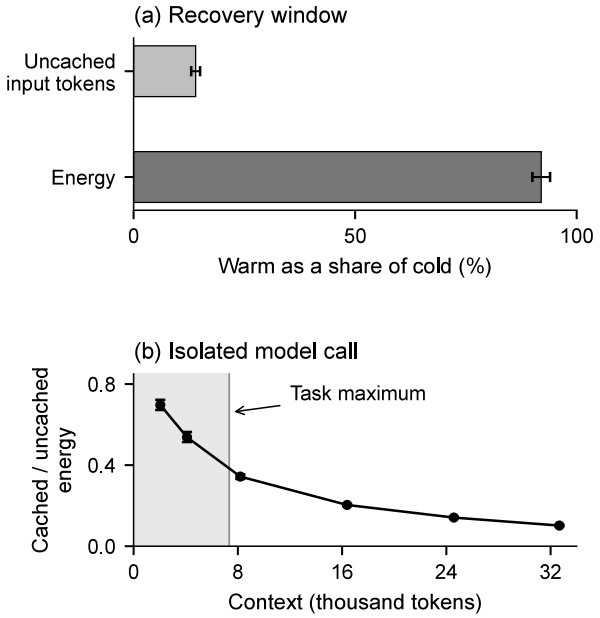

Figure 5. **Cache retention saves a larger share of uncached input tokens than of recovery energy.** (a) Uncached prompt tokens and energy of a warm recovery as a share of the same recovery cold, main study, with 95% bootstrap intervals over tasks. (b) Mean energy of an isolated cached call as a share of an uncached one, for Qwen3 30B on the same node, with 95% bootstrap intervals over calls at each context size. Lower ratios mean larger savings. Shading marks contexts up to the task maximum.

and rollback, this measure selects the same cheapest recovery as energy in three of the four combinations of failure side and point. In the fourth, the recovery with the lowest EpG needs under 1 percent more tokens than the recovery with the fewest tokens. After late tool failures, uncached-token counts favor restarting over resuming, while restarting has the highest energy per successful goal among these three warm recoveries (Table 2).

Finding 3. At contexts up to 7,333 tokens, keeping the cache warm removes 86 percent of a recovery's uncached prompt tokens but only 8 percent of its energy. After late tool failures, warm restart uses fewer uncached tokens than warm resume but has higher EpG (Figure 5, Table 2).

5.4 RQ4: Choosing a Recovery at the Failure

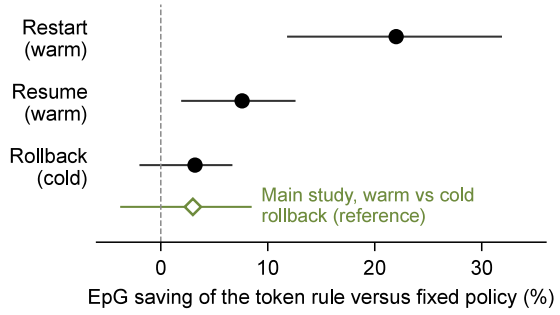

Figure 6. **The token rule lowers EpG relative to always restarting and always resuming.** Qwen3 30B, 27 complete held-out tasks, 108 goals per policy. The token rule selects the recovery predicted to need the fewest uncached prompt tokens. Points show percentage savings in pooled energy per successful goal over matched tasks. Bars are paired 95% bootstrap intervals over tasks. Positive values favor the rule. The fixed rollback comparator is cold. The open diamond gives, for reference, the main-study saving of warm over cold rollback for the same failure types and points.

A rule fitted on earlier experiments lowers EpG relative to always resuming and always restarting on held-out tasks, beyond the variation across tasks (Figure 6). The token rule uses the failure type and number of calls already made to predict each option's total uncached prompt tokens during recovery, then chooses the smallest prediction before the outcome is known. The token rule used 7.6 percent less energy per successful goal than always resuming (95% CI 2.0 to 12.5 percent). It used 22.0 percent less energy per successful goal than always restarting (11.9 to 31.8 percent), with 100 of 111 executed goals correct (Figure 6).

The rule chose warm rollback in 98 of 111 decisions and warm resume in the remaining 13 decisions. All resumes followed late failures, and 12 resumes followed malformed replies, where resuming is also cheapest in the main study. Because most choices are warm rollback, a fixed warm rollback is the closest simple comparator. The only rollback action executed as a fixed default on the held-out tasks was cold. Against that default, the rule used 3.2 percent less energy per successful goal (95% CI −1.9 to 6.6 percent, within the variation across tasks). In the main study, warm rollback used 3.0 percent less energy per successful goal than cold rollback for the same failure types and points (−3.7 to 8.4 percent). The rule's margin over cold rollback is close to this main-study saving, though the two estimates use different tasks. Executing fixed warm rollback on the held-out tasks would separate the benefit of choosing an operation from the benefit of retaining the cache.

Finding 4. On held-out tasks, the token rule uses 8 percent less energy per successful goal than always resuming and 22 percent less than always restarting, beyond the variation across tasks. It chose warm rollback in 98 of 111 decisions. Its estimated saving over always rolling back cold is within the variation across tasks (Figure 6).

6 Discussion

6.1 Choosing a Recovery in Practice

To choose between restarting and resuming with Equation 2, estimate the energy already spent, restart energy, success rates, and additional recovery energy on the intended tasks.

For agent frameworks. When the framework can restore both conversation and tool state, the main study favors warm rollback as a fixed recovery because it has the smallest mean excess cost among those tested (§5.2). The recovery with the lowest energy per successful goal depends on the model, tasks, and failure, so a framework should measure again when these change.

For benchmark and measurement designers. Benchmark designers should report energy per successful goal beside recovery success rate, and distinguish the conversation and tool state restored by recovery from the prompt computation reused through the prefix cache. They should also include failed goals and cached input when reporting tokens per successful goal (Section 5.3). Reports should state context lengths because the relative energy saving from cache retention grows with context.

6.2 Scope and Limitations

Failures and tasks. We study transient failures that reach the agent framework on multi-turn tasks the model solved reliably without injected failures. Rollback requires tools whose state the framework can restore. Tools with irreversible external effects require safeguards that coordinate recovery with those effects (Zheng et al., 2026; Wu et al., 2026a).

Measurement setting. We measure one goal at a time at the board of one GB10 node, above its idle baseline. Tool restoration, cache reset, and recovery decisions occur between measured attempt windows (Section 4). Slower restoration would add energy, which matters for close comparisons such as early rollback versus restart.

Models and serving. The comparisons describe recovery within each model's reliably solved tasks, and those task sets differ across models. Concurrent serving changes cache reuse and requires a separate evaluation.

Next comparisons. Restoring only the conversation or only the tools would separate their roles in rollback, following AgentRewind's tests of context and environment restoration (Zhuang et al., 2026). Executing fixed warm rollback on the held-out tasks would show whether choosing an operation lowers EpG beyond retaining the cache.

6.3 Reproducibility

The decision library includes the calibration files and logged decisions, and replaying it on the recorded inputs reproduces all 111 logged token-rule choices (Panigrahy et al., 2026a).

7 Conclusion

Agent frameworks should compare recoveries by the energy needed for correct results, including energy spent on failed runs. We compared eight recovery options after controlled call failures, with selected comparisons across three model families. On the main model, restarting lowers energy per successful goal after early failures, while resuming lowers it after late failures. Restarting's higher success rate and the additional energy used during resumption account for the early saving. Late in a task, preserving the work already completed makes resuming cheaper. Among the fixed recoveries tested on the main model, warm rollback has the smallest mean excess cost. At our task context lengths, a warm prefix cache saves a much larger share of uncached prompt tokens than of recovery energy. We suggest that framework developers choose how their agents recover by measuring energy per successful goal on their intended models and tasks, and that benchmarks report this cost beside recovery success rate. The cost of a correct result includes the energy of the runs that fail.

REFERENCES

Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., and Zhang, Y. Infercept: Efficient intercept support for augmented large language model inference, 2024. URL https://arxiv.org/abs/2402.01869.

Abhyankar, R., Qi, Q., and Zhang, Y. OSWorld-Human: Benchmarking the efficiency of computer-use agents. In *Proceedings of Machine Learning and Systems*, volume 8, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/5edb57c05c81d04beb716ef1d542fe9e-Abstract-Conference.html.

Anthropic. Prompt caching, 2025. URL https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching.

Cao, Z., Han, S., Zhang, J., Ren, Y., Li, Y., and Lee, P. P. C. Lumen: Coordinated failure recovery for distributed llm serving, 2026. URL https://arxiv.org/abs/2606.17787.

Chung, J.-W., Gu, Y., Jang, I., Meng, L., Bansal, N., and Chowdhury, M. Reducing energy bloat in large model training, 2024. URL https://arxiv.org/abs/2312.06902.

Chung, J.-W., Ma, J. J., Wu, R., Liu, J., Kweon, O. J., Xia, Y., Wu, Z., and Chowdhury, M. The ml.energy benchmark: Toward automated inference energy measurement and optimization, 2025. URL https://arxiv.org/abs/2505.06371.

Daly, J. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, February 2006. ISSN 0167-739X. doi: 10.1016/j.future.2004.11.016. URL https://doi.org/10.1016/j.future.2004.11.016.

Datadog. State of AI engineering 2026. Industry report, April 2026, 2026. URL https://www.datadoghq.com/about/latest-news/press-releases/datadog-state-of-ai-engineering-report-2026/.

Dubey, S. Real-time detection and repair of llm agent failures, 2026. URL https://arxiv.org/abs/2608.02464.

Efron, B. and Tibshirani, R. J. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994. doi: 10.1201/9780429246593. URL https://doi.org/10.1201/9780429246593.

Fortuna, C., Hanžel, V., Strnad, T., and Bertalanič, B. Where should agents live? energy-memory characterization of agentic ai for the edge-cloud continuum, 2026. URL https://arxiv.org/abs/2609.18283.

Gumaan, E. Feedback that backfires: Why small language model agents repeat the call they just watched fail, 2026. URL https://arxiv.org/abs/2608.23651.

Kang, H., Li, Z., Xu, W., Yang, X., Chen, Y., Wang, J., Chen, B., Krishna, T., Xu, C., and Arora, S. Thunderagent: A simple, fast and program-aware agentic inference system, 2026. URL https://arxiv.org/abs/2602.13692.

Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., and Narayanan, A. Ai agents that matter, 2024. URL https://arxiv.org/abs/2407.01502.

Khailo, M. Keeping the cache warm pays: Keepalive economics for agentic workloads, 2026. URL https://arxiv.org/abs/2607.19214.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

LangChain. LangGraph, 2024. URL https://github.com/langchain-ai/langgraph.

Li, H., He, R., Mang, Q., Zhang, Q., Mao, H., Chen, X., Zhou, H., Zhang, H., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live, 2026a. URL https://arxiv.org/abs/2511.02230.

Li, X., Yan, W., Wu, Y., Liang, P., Yuan, M., Liu, J., and Yang, J. Early diagnosis of wasted computation in multi agent LLM systems via failure aware observability, 2026b. URL https://arxiv.org/abs/2606.01365.

Luccioni, A. S., Jernite, Y., and Strubell, E. Power hungry processing: Watts driving the cost of ai deployment?, 2024. URL https://arxiv.org/abs/2311.16863.

Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., and Stoica, I. Autellix: An efficient serving engine for llm agents as general programs, 2025. URL https://arxiv.org/abs/2502.13965.

Luo, Q., Li, K., Wang, Z., and Chen, Y. Where does the energy go? profiling llm agent inference on blackwell gpus, 2026. URL https://arxiv.org/abs/2609.29707.

Mistral AI. Mistral small 3.2 24b instruct 2506. Model card, Hugging Face, mistralai/Mistral-Small-3.2-24B-Instruct-2506, 2025. URL https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506.

Mytkowicz, T., Diwan, A., Hauswirth, M., and Sweeney, P. F. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th international conference on Architectural support for programming languages and operating systems*, ASPLOS09, pp. 265–276. ACM, March 2009. doi: 10.1145/1508244.1508275. URL https://doi.org/10.1145/1508244.1508275.

NVIDIA. NVIDIA DGX Spark with the GB10 grace blackwell superchip, 2025. URL https://www.nvidia.com/en-us/products/workstations/dgx-spark/.

OpenAI. Prompt caching, 2025. URL https://platform.openai.com/docs/guides/prompt-caching.

OpenAI, Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., Arbus, E., Arora, R. K., Bai, Y., Baker, B., Bao, H., Barak, B., Bennett, A., Bertao, T., Brett, N., Brevdo, E., Brockman, G., Bubeck, S., Chang, C., Chen, K., Chen, M., Cheung, E., Clark, A., Cook, D., Dukhan, M., Dvorak, C., Fives, K., Fomenko, V., Garipov, T., Georgiev, K., Glaese, M., Gogineni, T., Goucher, A., Gross, L., Guzman, K. G., Hallman, J., Hehir, J., Heidecke, J., Helyar, A., Hu, H., Huet, R., Huh, J., Jain, S., Johnson, Z., Koch, C., Kofman, I., Kundel, D., Kwon, J., Kyrylov, V., Le, E. Y., Leclerc, G., Lennon, J. P., Lessans, S., Lezcano-Casado, M., Li, Y., Li, Z., Lin, J., Liss, J., Liu, L. X., Liu, J., Lu, K., Lu, C., Martinovic, Z., McCallum, L., McGrath, J., McKinney, S., McLaughlin, A., Mei, S., Mostovoy, S., Mu, T., Myles, G., Neitz, A., Nichol, A., Pachocki, J., Paino, A., Palmie, D., Pantuliano, A., Parascandolo, G., Park, J., Pathak, L., Paz, C., Peran, L., Pimenov, D., Pokrass, M., Proehl, E., Qiu, H., Raila, G., Raso, F., Ren, H., Richardson, K., Robinson, D., Rotsted, B., Salman, H., Sanjeev, S., Schwarzer, M., Sculley, D., Sikchi, H., Simon, K., Singhal, K., Song, Y., Stuckey, D., Sun, Z., Tillet, P., Toizer, S., Tsimpourlas, F., Vyas, N., Wallace, E., Wang, X., Wang, M., Watkins, O., Weil, K., Wendling, A., Whinnery, K., Whitney, C., Wong, H., Yang, L., Yang, Y., Yasunaga, M., Ying, K., Zaremba, W., Zhan, W., Zhang, C., Zhang, B., Zhang, E., and Zhao, S. gpt-oss-120b and gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Pan, Z., Patel, A., Hu, Z., Shen, Y., Guan, Y., Li, W.-L., Qin, L., Wang, Y., and Ding, Y. Kvflow: Efficient prefix caching for accelerating llm-based multi-agent workflows, 2025. URL https://arxiv.org/abs/2507.07400.

Panigrahy, D., Abkin, S., Tyagi, A., Houngninou, D. K., and Mahapatra, R. LLM agent recovery, 2026a. URL https://github.com/stephen122204/llm-agent-recovery.

Panigrahy, D., Tyagi, A., Abkin, S., Houngninou, D. K., and Mahapatra, R. Energy per successful goal: Goal-level energy accounting for agentic AI systems, 2026b. URL https://arxiv.org/abs/2605.22883.

Patel, P., Choukse, E., Zhang, C., Íñigo Goiri, Warrier, B., Mahalingam, N., and Bianchini, R. Polca: Power oversubscription in llm cloud providers, 2023. URL https://arxiv.org/abs/2308.12908.

Patel, P., Choukse, E., Zhang, C., Shah, A., Íñigo Goiri, Maleki, S., and Bianchini, R. Splitwise: Efficient generative llm inference using phase splitting, 2024. URL https://arxiv.org/abs/2311.18677.

Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large language model connected with massive apis, 2023. URL https://arxiv.org/abs/2305.15334.

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning (ICML)*, 2025. URL https://gorilla.cs.berkeley.edu/leaderboard.html.

Pham, D., Katevas, K., Shamsabadi, A. S., and Haddadi, H. Agentstop: Terminating local ai agents early to save energy in consumer devices, 2026. URL https://arxiv.org/abs/2605.15206.

Ruan, K., Huang, Z., Zhou, Z., Wei, Q., Lin, J., Wang, X., and Sun, H. Doomed from the start: Early abort of llm agent episodes via a recall-controlled probe cascade, 2026. URL https://arxiv.org/abs/2607.06503.

Samsi, S., Zhao, D., McDonald, J., Li, B., Michaleas, A., Jones, M., Bergeron, W., Kepner, J., Tiwari, D., and Gadepally, V. From words to watts: Benchmarking the energy costs of large language model inference, 2023. URL https://arxiv.org/abs/2310.03003.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

Strati, F., Mcallister, S., Phanishayee, A., Tarnawski, J., and Klimovic, A. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving, 2024. URL https://arxiv.org/abs/2403.01876.

Thompson, W. R. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933. doi: 10.2307/2332286.

Tian, Y., Sun, D., Ding, Y., and Liu, S. Cache your prompt when it's green: Carbon-aware caching for large language model serving, 2026. URL https://arxiv.org/abs/2505.23970.

vLLM Project. Automatic prefix caching, 2025. URL https://docs.vllm.ai/en/latest/features/automatic_prefix_caching.html.

Wang, C., Lyu, Y., He, J., Yang, Z., Zhong, C., Harel, Y., and Lo, D. Fail fast, restart smart: Early failure prediction and restart for SWE agentic tasks, 2026a. URL https://arxiv.org/abs/2608.03222.

Wang, Y., Yu, C., Zhang, J., Duan, Y., Zheng, M., Wu, Z., Shi, K., and Cai, T. From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents, 2026b. URL https://arxiv.org/abs/2608.10502.

Wu, G., Li, D., Jiang, K., Niu, J., Wang, C., and Zhang, Y. Safe to resume? breaking execution continuity of agent execution via rollback, 2026a. URL https://arxiv.org/abs/2608.29381.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

Wu, V., Han, J., Ruf, B., Grari, V., and Detyniecki, M. Prefill, decode, and context: A measured energy law for agentic LLM serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026b. URL https://neurips.cc/virtual/2026/170701.

Yan, F. A function calling perspective on scalable large language model agent evaluation. Master's thesis, EECS Department, University of California, Berkeley, 2025. URL https://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/31680.html.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Yang, Z. Why retrying fails: Context contamination in llm agent pipelines, 2026. URL https://arxiv.org/abs/2605.08563.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

You, J., Chung, J.-W., and Chowdhury, M. Zeus: Understanding and optimizing gpu energy consumption of dnn training, 2022. URL https://arxiv.org/abs/2208.06102.

Young, J. W. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17 (9):530–531, September 1974. ISSN 1557-7317. doi: 10.1145/361147.361115. URL https://doi.org/10.1145/361147.361115.

Yu, Y., Yao, L., Li, Y., Wang, E., and Wu, L. Rollback the world, keep the reflection: Rollback-induced reflection for long-horizon llm agents, 2026. URL https://arxiv.org/abs/2609.18304.

Zhang, Z. and Liu, W. Recoverability as a system primitive for long-horizon ai agents, 2026. URL https://arxiv.org/abs/2609.13672.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs, 2024. URL https://arxiv.org/abs/2312.07104.

Zheng, Y., Yang, Y., Zhang, W., and Quinn, A. Acrfence: Preventing semantic rollback attacks in agent checkpoint-restore, 2026. URL https://arxiv.org/abs/2603.20625.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024. URL https://arxiv.org/abs/2401.09670.

Zhuang, Y., Chen, K., Duan, Y., Zheng, S., Li, J., and Zhang, X.-Y. Agentrewind: Recoverable execution for long-horizon llm agents, 2026. URL https://arxiv.org/abs/2608.14380.

APPENDIX

A MEASUREMENT AND VALIDITY

The energy comparisons in Section 4 use the same measurement boundary across recoveries within each study and retain goals whose logs confirm the planned failure. A goal is one task run, including its initial execution and any recovery. Energy per successful goal (EpG) divides total measured goal energy, including failed goals, by the number that finish correctly. A warm prefix cache retains computed prompt state for reuse, while a cold cache clears it.

Energy boundary. The board's DC-input rail covers the whole node, including memory, storage, fans, and conversion losses outside the chip. We report board energy above the idle baseline measured with the model loaded and no requests. The package counter covers the chip, while NVIDIA Data Center GPU Manager (DCGM) reports the GPU alone. We read board and package power at 10 Hz and GPU power at 1 Hz, and check the ordering of these readings for each run. As a consistency check, we compare the sensor readings and find board energy above package energy and package energy above GPU energy. The earlier Qwen3 4B pilot study predates the board reading and uses the chip's package counter. In the main study, the time between attempt windows is at most 1.4 s. Every study ran vLLM 0.23.0 with prefix caching and chunked prefill on, at most 4 concurrent sequences and a maximum model length of 32,768 tokens.

Study coverage. Table 3 distinguishes the main comparison from studies that vary decoding, the model, or the kind of error. Sampling tests whether the ordering recurs when the model draws different replies, and the additional models use the tasks each solves reliably. Wrong-step runs test errors that the tool accepts, so the agent receives no error notice.

Injection and validity. The injector counts model calls and tool calls separately within the first attempt. Let n be the number of calls of the applicable kind in the first failure-free screening run. An early failure fires at the second call of that kind. A late failure fires at the rounded value of $0.7n$, clamped to $[3, n-1]$. The task configuration stores the resulting call numbers for model-side and tool-side failures. A wrong step replaces an argument of a tool call and counts only if the tool accepts it. A valid goal has exactly one firing, in the first attempt, at the planned failure type, position, and call number. An invalid goal is rerun once. The accepted-wrong-step summary additionally excludes cases marked self-corrected. Its early denominators range from 10 to 15 across recovery options, although the study inventory contains 26 tasks in total. The warm resume and warm restart counts reported in Section 5.1 each have 13 retained goals, with membership determined separately for each recovery. The counts describe those retained populations. A causal comparison of retaining the wrong step would require matching the task populations as well.

Table 3. Study configurations. All run on one GB10 node, one goal at a time. Goals include failure-free reference runs, and the Mistral and gpt-oss goals include failure-free screening of all 30 main-study tasks.

Study	Model	Tasks	Variation	Goals
Main	Qwen3 30B	30	4 failure types, 8 options	2,010
Sampling	Qwen3 30B	10	temperature 0.7, 3 seeds	240
Wrong steps	Qwen3 30B	26	accepted wrong step	239
Held-out	Qwen3 30B	28	rules and fixed recoveries	666
Mistral	Mistral 24B	8	3 recoveries	252
gpt-oss	gpt-oss 20B	12	3 recoveries	348
Pilot	Qwen3 4B	10	restart and resume	350

Shared history and the second batch. To assess how closely the experiments approximate a common state at failure, we check the histories of the initial attempts compared in Section 4. We compare the first attempts call by call, prompt and reply, after replacing generated call and request identifiers. The four options within each batch have identical calls and replies in 238 of 240 combinations of task, failure type, and point in the first batch and in 240 of 240 combinations in the second. Across batches, warm restart and warm resume each match warm rollback up to the failed call in 239 of 240 combinations. Across the 240 shared combinations, the second batch's initial attempts measured 5 percent more board energy than the first batch's. They ran about 2 percent longer, and the idle power subtracted for that batch was 0.5 to 0.8 W lower. As a sensitivity check, we divide second-batch energy by the pooled initial-attempt energy ratio. The observed winners and the direction of the resolved paired differences in Section 5.2 remain the same under this adjustment, which assumes that the ratio also applies to recovery attempts.

Paired confidence intervals. The intervals in Section 4 use 2,000 draws with replacement from the tasks shared by every recovery and failure condition in a comparison. Each draw uses the same tasks for both recoveries and all included failure conditions. A draw with zero successful goals for either recovery has undefined EpG and is omitted from the interval calculation. The resulting intervals describe variation across the selected tasks.

Isolated calls at long context. Figure 5b uses the Qwen3 30B sweep on the GB10 node, with 60 uncached and 60 cached calls across six context sizes up to 32,672 tokens. At each size, the ratio divides mean cached-call energy by mean uncached-call energy. The fits and ratios use 95% intervals from 1,000 resamples of calls with replacement, independently within each cache state and context size. The reported uncached fit has a positive quadratic coefficient, $7.81e-07$ J/token^2 (95% CI $7.01e-07$ to $8.74e-07$). Cached-call energy grows more slowly, with a fitted linear slope of 0.0023 J/token. This difference accompanies the decreasing cached-to-uncached ratio in the figure. These single-call measurements extend beyond the task contexts used for the recovery comparison.

A separate A100 sweep measures GPU energy only. Its positive quadratic coefficient is 0.546 J per (1k tokens)^2 (95% CI 0.508 to 0.584). That result supports the qualitative context dependence, but its hardware and measurement boundary differ from the GB10 board measurements. The task-level recovery comparisons of Section 5.3 use contexts only up to 7,333 tokens.

B Results by Failure Type and Point

The tables below support the recovery comparisons reported by model-call and tool-call failures in Section 5. Table 4 gives the main study's warm recoveries and cold rollback, including the lowest observed EpG in each failure condition. Table 5 gives the comparisons on the additional model families.

Warm restart and warm resume have the same early–late ordering for time per successful goal as for energy. After early failures, restart takes 20.3 s per successful goal and resume takes 25.9 s per successful goal. After late failures, resume takes 21.3 s per successful goal and restart takes 27.7 s per successful goal. In the token comparison (Section 5.3), cached input accounts for 91 to 94 percent of prompt tokens, so including it changes how tokens rank recoveries after late failures.

C The Cost Model

The condition in Section 3 compares energy per successful goal from a common state at failure. Let S be the energy already spent at failure, E the expected energy of a restart recovery, and $p_R > 0$ and $p_K > 0$ the success probabilities of restarting and resuming. The simplified cost model estimates unfinished work by $E - S$ and writes resumption energy as $E - S + \Delta$. Thus Δ is the residual above this estimate. The costs of restarting and resuming are

$$J_R = \frac{S+E}{p_R}, \quad J_K = \frac{S+(E-S)+\Delta}{p_K} = \frac{E+\Delta}{p_K}.$$

Multiplying $J_R < J_K$ by the positive product $p_R p_K$, we obtain $p_K(S+E) < p_R(E+\Delta)$, which rearranges to the condition in Equation 2,

$$S < \left(\frac{p_R}{p_K} - 1\right) E + \frac{p_R}{p_K}\Delta.$$

When $p_R = p_K$, the restart recovery energy E cancels and the condition becomes $S < \Delta$.

For rollback, let T be the energy of the completed portion of the interrupted turn that must be repeated and let $p_B > 0$ be its success probability. Assuming the remaining work otherwise has estimated energy $E - S$, total expected goal energy is $E + T$. With positive expected goal energy when resuming, $E + \Delta > 0$, we obtain

$$J_B = \frac{E+T}{p_B} < J_K \quad \Longleftrightarrow \quad \frac{p_B}{p_K} > \frac{E+T}{E+\Delta}.$$

If rollback and restart have equal success probability, $p_B = p_R$, rollback is cheaper than restarting when $T < S$. Rollback then repeats less completed work while preserving the same chance of success. These conditions compare recoveries once their energies and success rates are supplied. Choosing from call counts requires calibrating those quantities.

The terms reported in Section 5.1 are measured per failure type and point from the warm restart and warm resume goals of the main study. We estimate the common S by averaging the mean board energies of the initial attempts under restart and resume. The mean energy of a restart's recovery attempt gives E, and the observed success rates give p_R and p_K. We obtain Δ by subtracting $E - S$ from the mean energy of a resume's recovery attempt. The residual is positive for each tested failure type and point, so resumed execution uses more energy than this estimate of the remaining work. Substituting these measured terms reproduces the observed ordering when both recoveries use this shared estimate of S. This calculation accounts for recorded costs and outcomes. Prediction requires estimates available when the failure occurs.

D Held-Out Study

Table 6 supports the comparison in Section 5.4 between rules that choose a recovery during execution and fixed recovery choices.

The token fits use the main study and earlier experiments on wrong steps and failures that persist during recovery. Their tasks are disjoint from the held-out set. Each fit predicts uncached prompt tokens during recovery as an intercept plus a coefficient times the total model and tool calls made before failure. In the released decision records, the token rule chose warm rollback in 98 of 111 decisions and warm

Table 4. Main study (Qwen3 30B, 30 tasks). The ok columns count correct goals. J/run gives mean energy per run, including failed runs, and EpG gives energy per successful goal, both in board joules. Bold marks the lowest observed EpG of all eight recovery options in a row. The displayed warm recoveries and cold rollback include the lowest option in each row.

		Restart, warm			Resume, warm			Rollback, warm		Rollback, cold		Grounded, warm	
Type	Point	ok	J/run	EpG	ok	J/run	EpG	ok	EpG	ok	EpG	ok	EpG
Malformed reply	early	28/30	1,231	**1,319**	22/30	1,403	1,914	23/30	1,437	22/30	1,641	23/30	1,754
Rate limit	early	28/30	1,173	**1,257**	25/30	1,264	1,517	23/30	1,332	22/30	1,526	25/30	1,482
Tool error	early	28/30	1,267	1,357	22/30	1,248	1,702	26/30	**1,314**	26/30	1,367	25/30	1,589
Timeout	early	28/30	1,258	1,347	22/30	1,237	1,686	26/30	**1,312**	26/30	1,382	25/30	1,587
Malformed reply	late	28/30	1,784	1,912	28/30	1,245	**1,334**	26/30	1,561	28/30	1,527	27/30	1,468
Rate limit	late	28/30	1,691	1,812	28/30	1,161	**1,244**	26/30	1,471	28/30	1,465	27/30	1,354
Tool error	late	28/30	1,711	1,833	24/30	1,208	1,510	27/30	1,439	29/30	**1,421**	26/30	1,543
Timeout	late	28/30	1,692	1,812	24/30	1,207	1,508	27/30	1,471	29/30	**1,447**	26/30	1,585

Table 5. Mistral Small 3.2 24B (8 tasks) and gpt-oss 20B (12 tasks), each on the tasks it solves reliably. The ok columns count correct goals, and EpG gives board joules per successful goal for warm restart, warm resume, and cold rollback. Bold marks the lowest observed EpG in a row.

		Restart		Resume		Rollback	
Type	Point	ok	EpG	ok	EpG	ok	EpG
Mistral Small 3.2 24B							
Malformed	early	8/8	**4,887**	6/8	6,785	7/8	5,735
Rate limit	early	7/8	**5,078**	6/8	5,812	6/8	6,216
Tool error	early	8/8	4,984	7/8	**4,717**	7/8	5,380
Timeout	early	8/8	5,038	7/8	**4,775**	7/8	5,531
Malformed	late	8/8	7,144	8/8	**4,330**	7/8	5,201
Rate limit	late	8/8	6,912	6/8	6,244	7/8	**5,026**
Tool error	late	8/8	7,013	7/8	**4,990**	7/8	5,323
Timeout	late	7/8	7,921	7/8	**4,660**	7/8	5,311
gpt-oss 20B							
Malformed	early	11/12	2,848	6/12	5,648	11/12	**2,842**
Rate limit	early	10/12	**2,725**	8/12	3,974	9/12	4,970
Tool error	early	12/12	**2,428**	9/12	3,781	8/12	3,960
Timeout	early	10/12	**3,690**	6/12	5,453	9/12	4,158
Malformed	late	10/12	5,307	11/12	**2,559**	9/12	3,581
Rate limit	late	12/12	3,410	10/12	2,666	11/12	**2,564**
Tool error	late	11/12	3,701	12/12	**2,226**	10/12	3,856
Timeout	late	10/12	5,336	11/12	**2,409**	11/12	2,867

resume in 13 (Panigrahy et al., 2026a). The 13 resumes followed late failures, including 12 malformed replies and one tool error. The predictions placed warm rollback below warm resume in 98 decisions, with a median gap of 69 uncached tokens across all decisions. The fitted predictions determine these choices for individual tasks and failures. A fixed warm-rollback choice would agree with the rule in 98 cases. Measuring its energy and correctness on these tasks requires executing that fixed choice as well.

The learning controller represents uncertainty in success with a Beta distribution for each failure type and recovery, then draws one success rate for each recovery before choosing. It scores each recovery by its predicted recovery energy divided by that sampled rate. Equation 1 also includes energy spent before failure, which affects comparisons when success rates differ. Below its restart threshold, it considered only restart options in 56 of its 111 recorded decisions. Its fitted parameters came from the calibration preceding the correction to how repeated goals were matched across studies. The energy predictions also used zero token counts. Supplying the recorded token counts in a replay diagnostic changes 44 of 111 choices at the first recovery attempt. These settings define the learning-controller configuration evaluated here. The threshold rule used 16.0 percent less energy per successful goal than always restarting. The learning controller used 13.3 percent less energy per successful goal than always restarting. Table 7 supports the fixed-recovery comparison in Section 5.2, giving each fixed recovery's excess EpG relative to the cheapest option for each failure type and point, identified after observing outcomes. Because that reference is selected from the same data, it gives an optimistic comparison. The released library includes a replay test that reproduces the 111 recorded token-rule

Table 6. Held-out study (Qwen3 30B, 28 tasks, 111 decisions per policy). The ok column counts correct goals, J/run gives mean energy per run including failed runs, and EpG gives energy per successful goal, both in board joules. The paired savings use 27 complete tasks (108 goals per policy), so they differ from ratios of these all-goal EpGs. Bold marks the lowest observed EpG, whose paired difference from always rollback (cold) has an interval that includes zero (Section 5.4).

Policy	ok	J/run	EpG
Learning controller	95/111	1,248	1,459
Threshold rule	98/111	1,242	1,406
Token rule	100/111	1,167	**1,296**
Always restart (warm)	101/111	1,512	1,662
Always resume (warm)	97/111	1,236	1,414
Always rollback (cold)	101/111	1,231	1,353

choices (Panigrahy et al., 2026a). Evaluating their energy and correctness requires the experimental records alongside those choices.

Threshold calibration. The thresholds supplied to the rule range from 3.9 to 8.1 calls across failure types and retain the earlier calibration. The held-out study measures how that rule performs during execution. Validating the call thresholds against Equation 2 requires fitting them to its energy and success-rate terms.

Table 7. Extra energy per successful goal of a fixed recovery relative to the cheapest recovery for each failure type and point, identified after observing outcomes (percent, board). The first three columns summarize each study's measured recoveries. The held-out column gives measured executions relative to the best of its three fixed recovery choices. Entries average over eight failure conditions in the main and additional-model studies and four in the held-out study. A dash marks an untested recovery, and bold marks the smallest excess on Qwen3 30B.

Policy	Qwen3 30B	Mistral	gpt-oss	Held-out
Restart (warm)	18.7	27.3	36.7	25.5
Resume (warm)	15.2	8.7	23.8	6.8
Rollback (warm)	**6.5**	–	–	–
Rollback (cold)	10.1	13.6	32.8	2.2